

F: Finding Common Passwords

Encontrando contraseñas comunes

Nombre del archivo fuente (*Source file name*): `find.c`, `find.cpp`, `find.java` o `find.py`

Autor (*Author*): Andrés Mejía

Como ingeniero de seguridad (*Security Engineer*), una de tus principales responsabilidades es asegurarte de que los empleados de la empresa usen contraseñas poco comunes y difíciles de adivinar. Desafortunadamente, sospechas que tus colegas no se están ciñendo a esta idea: varias cuentas de correo electrónico, con información confidencial de la empresa, han sido comprometidas recientemente por un grupo de hackers conocido como International Common Password Connoisseurs (ICPC; «conocedores internacionales de contraseñas comunes»).

Para protegerse de futuros ataques, comenzaste una auditoría de una muestra anonimizada de contraseñas, y descubriste que la gente con frecuencia incluye palabras comunes como `admin` o `secret` en sus contraseñas. Resulta, por ejemplo, ¡que `qwerty` apareció en al menos el 60 % de las contraseñas! Decidiste buscar sistemáticamente cadenas largas que aparezcan con frecuencia en la lista de contraseñas. Concretamente, dada una lista de N contraseñas y un entero positivo K , buscas la cadena más larga de las N contraseñas que aparezca en al menos K de ellas.

Por ejemplo, considera la siguiente lista de $N = 5$ contraseñas:

`monkey` `monk` `money` `motorcycle` `recycle`

En el caso $K = 2$, la (sub)cadena más larga con esa propiedad es `cycle`. En el caso $K = 3$, la subcadena más larga es `mon`. Sin embargo, en el caso $K = 5$, la subcadena más larga es la vacía.

Recuerda que una subcadena (*substring*) se define como una secuencia contigua de caracteres dentro de una cadena. Por ejemplo, `monk`, `onke` y `key` son subcadenas de `monkey`, pero `money` no lo es. Por definición, una cadena siempre es subcadena de sí misma. Observa que, en algunos casos, puede haber varias respuestas. En este problema, en tales casos, interesa la subcadena lexicográficamente menor (*lexicographically smallest*), es decir, la que aparecería primero en un diccionario. Por ejemplo, `aac` es lexicográficamente menor que `aacb`, `ab` y `aad`, pero `aac` no es lexicográficamente menor que `aab`.

Dada una lista de N contraseñas y un entero K , ¿cuál es la cadena más larga de las N contraseñas que aparece en al menos K de ellas?

Entrada (Input)

La entrada consiste en varios casos de prueba. Cada caso de prueba comienza con una línea con dos enteros N y K ($1 \leq N \leq 100\,000$ y $1 \leq K \leq N$). Las siguientes N líneas contienen la lista de contraseñas, una por línea. Las contraseñas son no vacías y solo contienen letras minúsculas del alfabeto inglés (a-z). La suma de las longitudes de todas las contraseñas de la lista no excede 100 000. La última línea de la entrada contiene dos ceros separados por un espacio y no debe procesarse.

La entrada debe leerse de la entrada estándar.

Salida (Output)

Para cada caso de prueba, imprime una sola línea con la cadena más larga que sea subcadena de al menos K contraseñas. Si hay varias opciones posibles, imprime la que sea lexicográficamente menor. Si dicha cadena es la vacía, imprime un solo asterisco (*).

La salida debe escribirse en la salida estándar.

Ejemplo de entrada (Sample Input)	Ejemplo de salida (Sample Output)
5 1 monkey monk money motorcycle recycle 5 2 monkey monk money motorcycle recycle 5 3 monkey monk money motorcycle recycle 5 4 monkey monk money motorcycle recycle 5 5 monkey monk money motorcycle recycle 3 1 security security insecure 3 2 security security insecure 3 3 security security insecure 0 0	motorcycle cycle mon mo * insecure security secur